

CRYPTO HAWKING · ETHDENVER 2025 WINNER

Smart Contract Security Audit

WstETH

0x7f39C581F595B53c5cb19bD0b3f8dA6c935E2Ca0

Ethereum

Compiler: v0.6.12+commit.27d51765

Date: October 7, 2026

OVERALL RISK RATING AI-assisted audit. Not a substitute for manual review.

Medium

Executive Summary

Independent security review of publicly verified source, not commissioned by the project. WstETH is Lido's canonical ERC20Permit wrapper around rebasing stETH, converting between static wstETH shares and pooled ETH value via the trusted stETH contract. The implementation follows checks-effects-interactions ordering (mint/burn before external calls) and contains no owner-gated or upgradeable attack surface. The main findings relate to unchecked external call return values on the stETH integration and minor hardening/gas improvements; no critical or high-severity issues were identified given the fully trusted, immutable dependency on the official stETH contract.

Findings (2)

F-001 · SWC-104

Medium

Unchecked return value of stETH.transfer/transferFrom calls

WstETH.sol:228–243

DESCRIPTION

In `wrap()` and `unwrap()`, the return values of `stETH.transferFrom(...)` and `stETH.transfer(...)` are not checked. The current stETH implementation reverts on failure rather than returning `false`, which masks the issue today, but the code does not defensively verify success, violating the checked-call pattern recommended for ERC20 interactions.

IMPACT

If the referenced stETH implementation (or any future token with similar interface semantics) were to return `false` instead of reverting on a failed transfer, wstETH could mint/burn tokens without the corresponding stETH actually moving, resulting in an accounting mismatch between the two contracts.

RECOMMENDATION

Wrap external transfer calls with `require(...)` or use OpenZeppelin's `SafeERC20.safeTransfer` / `safeTransferFrom` to guarantee calls revert on failure regardless of the token's return-value convention:

```
```solidity
require(stETH.transferFrom(msg.sender, address(this), _stETHAmount), "wstETH: transferFrom failed");
...
require(stETH.transfer(msg.sender, stETHAmount), "wstETH: transfer failed");
```
```

```
function wrap(uint256 _stETHAmount) external returns (uint256) {
    require(_stETHAmount > 0, "wstETH: can't wrap zero stETH");
    uint256 wstETHAmount = stETH.getSharesByPooledEth(_stETHAmount);
    _mint(msg.sender, wstETHAmount);
    stETH.transferFrom(msg.sender, address(this), _stETHAmount);
    return wstETHAmount;
}
```

F-002

Low

Missing zero-address validation for stETH address in constructor

WstETH.sol:184–191

DESCRIPTION

The constructor accepts an `IStETH \_stETH` address and stores it without validating that it is non-zero or a contract address.

IMPACT

Deploying the contract with an incorrect or zero stETH address would render the contract non-functional (all calls to `stETH.\*` would revert against `address(0)`), requiring a full redeploy since `stETH` has no setter.

RECOMMENDATION

Add a zero-address check in the constructor:

```
```solidity
constructor(IStETH _stETH) public ERC20Permit(...) ERC20(...) {
 require(address(_stETH) != address(0), "wstETH: zero stETH address");
 stETH = _stETH;
}
```
```

```
constructor(IStETH _stETH)
public
ERC20Permit("Wrapped liquid staked Ether 2.0")
ERC20("Wrapped liquid staked Ether 2.0", "wstETH")
{
  stETH = _stETH;
}
```

Gas Optimizations

Declare `stETH` as `immutable`

WstETH.sol:183–undefined

Since `stETH` is only ever set once in the constructor and never updated, declaring it `immutable` would save an SLOAD on every read across `wrap`, `unwrap`, `receive`, and the view helper functions, reducing gas costs for all users.

Cache `stETH` storage variable in a local variable within functions with multiple reads

WstETH.sol:226–241

In `wrap()` and `unwrap()`, `stETH` is read from storage multiple times (once for the share calculation call, once for the transfer call). Caching it in a local variable (`IStETH _s = stETH;`) at the top of each function avoids redundant SLOADs when not combined with the immutable fix above.

Appendix - Metadata

| | |
|----------|--|
| Contract | WstETH |
| Address | 0x7f39C581F595B53c5cb19bD0b3f8dA6c935E2Ca0 |
| Chain | Ethereum |
| Compiler | v0.6.12+commit.27d51765 |
| Audit ID | portfolio-lido-wsteth |
| Date | 10/7/2026 |

Files analyzed

[user] WstETH.sol

MANUAL AUDIT UPGRADE

Need a deeper audit?

\$5,000

Pay in ETH, SOL, or USDT. Detailed manual review by Crypto Hawking, ETHDenver 2025 Winner. Comprehensive findings, remediation steps, and post-fix re-review. Report delivered in 3 business days.

[Request Manual Audit '](https://cryptohawking.com/audit/manual?ref=portfolio-lido-wsteth)

<https://cryptohawking.com/audit/manual?ref=portfolio-lido-wsteth>

Need it built, not just audited?

I also build production smart contracts end to end, token launches, DeFi protocols, staking, and full DEX stacks (v2, v3 concentrated liquidity, and beyond), plus fractional CTO engagements for Web3 teams. Fixes for the findings in this report can be scoped in the same engagement.

<https://cryptohawking.com/services>

WhatsApp (preferred): +92 322 4274236, <https://wa.me/923224274236>